

UNIVERSITÉ MOULAY ISMAIL

Faculté des Sciences de Meknès

Département d'Informatique

Parcours d'Excellence — Master SDIA 2025–2026

Rapport d'État d'Avancement Sprint 1

Agentic GraphRAG pour le Raisonnement Complexe sur des Corpus Techniques

SPRINT 1 — Fondations
État de l'Art, Environnement
& Préparation des Données
07 Avril – 04 Mai 2026

Étudiante : Wiame Anejjar

Encadrant : Pr. Abdelaaziz Hessane

Année : 2025–2026

Date : 04 Mai 2026

Table des matières

Résumé Exécutif	6
1 Introduction et Contexte du Projet	8
1.1 Problématique	8
1.2 Objectifs du Projet	8
1.3 Valeur Ajoutée et Contribution Scientifique	9
2 État de l’Art : RAG, GraphRAG et Agentic RAG	10
2.1 Retrieval-Augmented Generation (RAG)	10
2.1.1 Contexte et Motivation	10
2.1.2 Fonctionnement du RAG Classique	10
2.1.2.1 Étape 1 : Indexation	11
2.1.2.2 Étape 2 : Récupération	11
2.1.2.3 Étape 3 : Génération	11
2.1.3 Paradigmes du RAG et métriques d’évaluation	12
2.1.4 Limites du RAG Classique pour le Raisonnement Multi-Sauts	13
2.2 GraphRAG : Intégration des Graphes de Connaissances	14
2.2.1 Motivation et Concept	14
2.2.2 Pipeline GraphRAG Microsoft	14
2.2.3 Performances de GraphRAG	15
2.2.4 Limitation Critique de GraphRAG Microsoft	15
2.2.5 LightRAG : Une Alternative Économique	16
2.2.5.1 Architecture Dual-Level de LightRAG	16
2.2.5.2 Avantages de LightRAG sur GraphRAG Microsoft	16
2.3 Agentic RAG : Notre Contribution Principale	17
2.3.1 Concept et Motivation	17
2.3.2 Architecture de notre Agent LangGraph	17
2.3.2.1 Nœud 1 : QUERY_ANALYSIS	17
2.3.2.2 Nœud 2 : GRAPH_SEARCH	17
2.3.2.3 Nœud 3 : VECTOR_SEARCH	17
2.3.2.4 Nœud 4 : CRITIQUE	18

2.3.2.5	Nœud 5 : SYNTHESIZE	18
2.3.2.6	Nœud 6 : SELF_CORRECT	18
2.3.3	LangGraph : Framework de l'Agent	18
2.3.3.1	Différence LangGraph vs LangChain	19
2.4	Tableau Comparatif : RAG vs GraphRAG vs Agentic RAG	20
3	Choix du Dataset et Justification	22
3.1	Critères de Sélection	22
3.2	Corpus : arXiv CS.AI	22
3.2.1	Description	22
3.2.2	Justification du Choix	22
3.2.3	Analyse de corpus arXiv CS.AI	23
3.3	Benchmark : HotpotQA	23
3.3.1	Description	23
3.3.2	Types de Questions	23
3.3.3	Justification du Choix de HotpotQA	24
3.4	Statistiques Descriptives du Dataset	24
4	Environnement Technique	26
4.1	Vue d'ensemble de la stack technologique	26
4.2	Installation et Configuration	27
4.2.1	Python 3.12 et environnement virtuel	27
4.2.2	Ollama et Llama-3.1-8B	27
4.2.3	Neo4j via Docker	28
4.3	Structure du Projet	29
5	Travaux Réalisés	30
5.1	Travail 1 : Prétraitement du Corpus	30
5.1.1	Objectif	30
5.1.2	Pipeline de Nettoyage	30
5.1.3	Résultats	31
5.2	Travail 2 : Chunking avec Overlap	32
5.2.1	Objectif et Paramètres	32
5.2.2	Mécanisme Sliding Window	33
5.2.3	Résultats et Visualisation	34
5.3	Travail 3 : Indexation LightRAG	34
5.3.1	Architecture LLM + Embedding	34
5.3.2	Système de Checkpoint	36
5.3.3	Export des Chunks vers ChromaDB	36
5.3.4	Résultats de l'Indexation	37

5.4	Travail 4 : Test des Modes Naive et Hybrid	37
5.4.1	Description	37
5.4.2	Résultats Obtenus	37
5.5	Travail 5 : Export vers Neo4j et Analyse du Graphe	38
5.5.1	Pipeline d'Export	38
5.5.2	Résultats et Visualisation du Graphe	38
5.6	Problèmes rencontrés et solutions	39
5.6.1	Problèmes liés à la collecte et au traitement des données	39
5.6.2	Problèmes d'indexation et de ressources LLM	39
5.6.3	Problèmes liés à la qualité du graphe	40
5.6.4	Solutions adoptées	40
5.7	Conclusion et plan du Sprint 2	40

Table des figures

2.1	Pipeline du RAG classique : indexation, récupération et génération . . .	11
2.2	Pipeline GraphRAG	15
2.3	Architecture cible de l'agent Agentic GraphRAG	18
3.1	Distribution des tokens	23
3.2	Distribution thèmes	23
3.3	Analyse de benchmark HotpotQA	24
3.4	Statistiques Descriptives de Corpus arXiv CS.AI	24
3.5	Statistiques Descriptives de Benchmark HotpotQA	24
3.6	Statistiques Descriptives du Dataset	25
4.1	Validation d'Ollama via curl : Llama-3.1-8B répond correctement . . .	28
4.2	Conteneur Neo4j actif dans Docker Desktop	28
4.3	Structure du projet dans VSCode	29
5.1	Fonction de nettoyage du texte	31
5.2	Résultats du pipeline de nettoyage	31
5.3	Algorithme de chunking avec overlap	33
5.4	Résultats du chunking avec overlap	34
5.5	Distribution des tokens par chunk et schéma du sliding window avec overlap	34
5.6	Fonction Groq avec retry automatique	35
5.7	Fonction d'embedding Ollama : retour NumPy 2D	36
5.8	Terminal montrant le déroulement de l'indexation LightRAG : 100 docu- ments indexés avec succès	37
5.9	Graphe de connaissances visualisé dans Neo4j Browser	39

Liste des tableaux

- 2.1 Comparaison détaillée des paradigmes RAG selon [2] 12
- 2.2 Win rate de GraphRAG vs RAG classique [3] 15
- 2.3 Comparaison LightRAG vs GraphRAG Microsoft 16
- 2.4 LangGraph vs LangChain 19
- 2.5 Comparaison complète des trois approches 20

- 4.1 Stack technologique complète du projet 27

- 5.1 Comparaison Naive vs Hybrid 38
- 5.2 Statistiques du graphe de connaissances Sprint 1 38

Résumé Exécutif

Sprint 1 — Vue d'ensemble

Période : 07 Avril – 04 Mai 2026 (4 semaines)

Thème : Fondations — État de l'Art, Environnement & Préparation des Données

Statut : Terminé

Ce qui était prévu

Le Sprint 1 avait pour objectif d'établir les fondations théoriques et techniques du projet Agentic GraphRAG. Les tâches prévues étaient :

- (1) la lecture et l'analyse des 7 articles clés de la littérature.
- (2) l'installation de l'environnement complet (Docker, Ollama, Neo4j, Python).
- (3) le téléchargement et le prétraitement du corpus de données.
- (4) l'indexation test de 100 documents dans LightRAG avec analyse du graphe généré dans Neo4j Browser.

Ce qui a été fait

L'ensemble des tâches prévues a été réalisé avec succès.

- Les 7 articles fondateurs ont été lus et synthétisés.
- L'environnement technique complet a été configuré et validé.
- Un corpus de 500 abstracts arXiv CS.AI a été téléchargé via l'API officielle (sans PDF, conformément aux recommandations de l'encadrant).
- Le pipeline de prétraitement et de chunking a été implémenté.
- L'indexation de 100 documents test dans LightRAG a été réalisée avec succès, produisant un graphe de connaissances de 1355 nœuds et 620 relations, exporté et analysé dans Neo4j Browser.
- Plusieurs problèmes techniques significatifs ont été rencontrés et résolus, contribuant à une compréhension approfondie de la pile technologique.

Ce qui reste à faire

- Améliorer le pipeline d'indexation existant.
- Passer au Sprint 2, qui nécessite les tâches suivantes :
 - Étudier en profondeur l'API LightRAG (modes : naive, local, global, hybrid) et indexer 100% du corpus en mode hybrid, puis logger tous les triplets extraits dans un fichier JSON pour analyse.
 - Implémenter une extraction NER+RE complémentaire avec spaCy + Rebel ou GLiNER, et comparer qualitativement les triplets obtenus avec ceux de LightRAG, puis Valider manuellement 50 triplets et calculer le taux de précision.
 - Charger le graphe complet dans Neo4j, et écrire 10 requêtes Cypher exploratoires : entités les plus connectées, chemins entre deux entités, sous-graphes thématiques, détection de communautés.
 - Implémenter le RAG vectoriel baseline (LangChain + ChromaDB + Llama, et tester sur 20 questions HotpotQA, puis documenter précisément les échecs du RAG classique sur les questions multi-sauts.

Chapitre 1

Introduction et Contexte du Projet

1.1 Problématique

Les Grands Modèles de Langage (LLM) révolutionnent le traitement automatique du langage naturel. Cependant, leur déploiement dans des contextes professionnels soulève des défis fondamentaux que les approches actuelles peinent à résoudre efficacement.

Le premier défi est celui des **hallucinations** : les LLM génèrent parfois des informations factuellement incorrectes avec une apparente confiance [8]. Le deuxième est l'**obsolescence des connaissances** : les paramètres du modèle, figés à l'entraînement, ne reflètent pas les découvertes récentes. Le troisième est le **raisonnement multi-sauts** : répondre à des questions complexes nécessitant la traversée de plusieurs documents et l'intégration d'informations hétérogènes dépasse les capacités des systèmes RAG classiques.

Un autre enjeu majeur concerne la confidentialité des données : de nombreuses entreprises refusent d'exposer leurs données sensibles à des LLM externes. Elles ont besoin de solutions de chatbot personnalisées, capables d'exploiter leurs données privées de manière sécurisée, tout en étant suffisamment intelligentes pour répondre à des questions variées : simples, complexes ou comparatives spécifiques à leur domaine.

Question de recherche centrale

Comment concevoir un système capable de répondre avec précision à des questions complexes nécessitant un raisonnement multi-sauts sur un corpus technique dense, en combinant un graphe de connaissances et un agent de raisonnement autonome ?

1.2 Objectifs du Projet

Dans ce contexte, ce projet de fin d'études propose une solution aux problématiques identifiées en visant à concevoir, implémenter et évaluer un système Agentic GraphRAG

capable de répondre à des questions complexes nécessitant un raisonnement multi-sauts sur des corpus techniques.

Les objectifs spécifiques sont :

1. Construire un graphe de connaissances structuré à partir d'un corpus technique en utilisant LightRAG
2. Développer un agent autonome basé sur LangGraph capable de raisonnement multi-sauts itératif
3. Évaluer quantitativement le système sur le benchmark HotpotQA avec les métriques RAGAS
4. Comparer les performances de l'approche Agentic GraphRAG face au RAG classique et au GraphRAG

1.3 Valeur Ajoutée et Contribution Scientifique

La contribution principale de ce PFE est la **couche agentique** construite au-dessus de LightRAG. Notre agent LangGraph introduit trois capacités nouvelles :

- **Auto-évaluation** : le nœud CRITIQUE évalue si les informations récupérées sont suffisantes pour répondre à la question
- **Raisonnement itératif** : le nœud SELF_CORRECT reformule les requêtes et relance la recherche si nécessaire
- **Stratégie adaptative** : le nœud QUERY_ANALYSIS décompose les questions complexes en sous-questions et choisit intelligemment entre recherche graphe et recherche vectorielle

Chapitre 2

État de l'Art : RAG, GraphRAG et Agentic RAG

2.1 Retrieval-Augmented Generation (RAG)

2.1.1 Contexte et Motivation

Avant l'émergence du RAG, l'adaptation des LLM à des domaines spécifiques reposait exclusivement sur le **fine-tuning** : modifier les paramètres du modèle via un entraînement supplémentaire sur des données de domaine. Cette approche présente trois limitations majeures [2] :

- **Coût élevé** : plusieurs heures à plusieurs jours de calcul GPU
- **Obsolescence** : le modèle ne peut pas intégrer de nouvelles informations sans être réentraîné
- **Opacité** : impossible de tracer l'origine d'une réponse générée

Le RAG, introduit par [1], propose une alternative élégante : au lieu de modifier les paramètres du modèle, on lui fournit dynamiquement les informations pertinentes au moment de la requête.

2.1.2 Fonctionnement du RAG Classique

Le RAG classique, aussi appelé Naive RAG dans la littérature [2], suit un pipeline en trois étapes illustré dans la figure suivante :

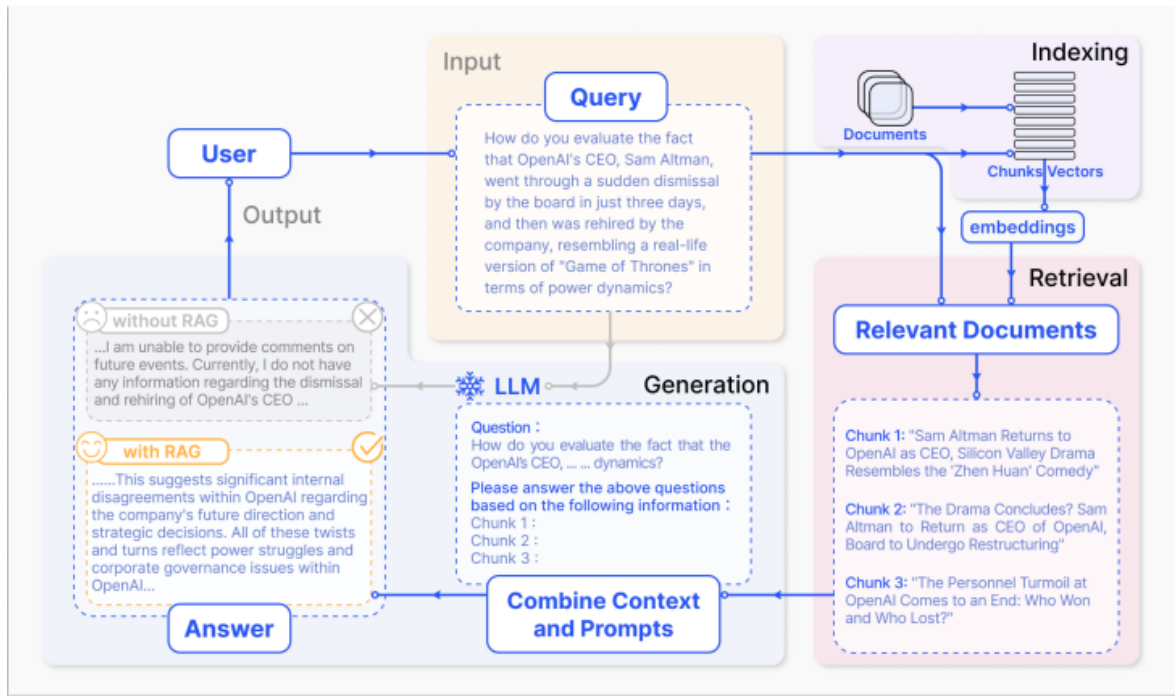


FIGURE 2.1 – Pipeline du RAG classique : indexation, récupération et génération

2.1.2.1 Étape 1 : Indexation

Lors de la phase d'indexation (offline), les documents du corpus sont :

1. **Découpés en chunks** : segments de texte de taille fixe (typiquement 256–1024 tokens)
2. **Encodés en vecteurs** : via un modèle d'embedding (*Dense Passage Retrieval* [7], *BGE*, *nomic-embed-text*)
3. **Stockés dans une base vectorielle** : ChromaDB, Pinecone, Weaviate, FAISS

2.1.2.2 Étape 2 : Récupération

À l'arrivée d'une question utilisateur, le système :

1. Encode la question avec le même modèle d'embedding
2. Calcule la similarité cosinus entre le vecteur question et tous les vecteurs du corpus
3. Sélectionne les k chunks les plus similaires (typiquement $k = 5$ à 20)

2.1.2.3 Étape 3 : Génération

La question et les k chunks récupérés sont concaténés dans un prompt envoyé au LLM, qui génère la réponse finale en se basant sur ces informations contextuelles.

2.1.3 Paradigmes du RAG et métriques d'évaluation

La revue de littérature de [2] identifie trois paradigmes d'évolution du RAG :

Paradigme	Naive RAG	Advanced RAG	Modular RAG
Indexation	Découpage simple du corpus en segments fixes (chunks) de taille uniforme, puis transformation en vecteurs via un modèle d'embedding standard, sans enrichissement contextuel.	Utilisation de techniques plus avancées comme le <i>sliding window</i> pour préserver le contexte, ajout de métadonnées (titre, source, etc.) et amélioration des embeddings via fine-tuning.	Architecture flexible où l'indexation est composée de modules indépendants et configurables (choix du découpage, des embeddings, des métadonnées), adaptés au cas d'usage.
Récupération	Recherche basée uniquement sur la similarité cosinus entre la requête et les vecteurs du corpus, sans optimisation supplémentaire.	Ajout de mécanismes comme le <i>reranking</i> pour réordonner les résultats, la compression du contexte pour réduire le bruit, et des techniques comme HyDE pour enrichir la requête.	Récupération dynamique et adaptative, pouvant combiner plusieurs index (vectoriel, lexical, graphe) et ajuster la stratégie selon la question posée.
Génération	Le LLM génère directement une réponse à partir des chunks récupérés, sans traitement supplémentaire.	Intégration d'étapes de post-traitement (filtrage, résumé, compression) pour améliorer la qualité et la pertinence de la réponse générée.	Utilisation de modules spécialisés (agents, outils externes, self-RAG) permettant un raisonnement plus complexe et itératif avant de produire la réponse finale.
Limitations	Difficulté à traiter des questions complexes nécessitant un raisonnement multi-sauts, sensibilité au bruit et aux chunks non pertinents.	Amélioration des performances mais au prix d'une complexité accrue, d'un coût computationnel plus élevé et d'une mise en œuvre plus délicate.	Grande flexibilité mais forte complexité d'implémentation et de maintenance, nécessitant une orchestration avancée des modules.

TABLE 2.1 – Comparaison détaillée des paradigmes RAG selon [2]

Dans ce cadre, notre projet s'inscrit dans le paradigme du Modular RAG, en proposant une architecture Agentic GraphRAG composée de modules spécialisés (indexation, extraction de connaissances, graphe, récupération hybride et raisonnement agentique).

Métriques d'évaluation :

Les métriques d'évaluation utilisées pour le RAG, définies dans le framework RAGAS [6], sont :

- **Faithfulness** : la réponse est-elle fidèle aux sources récupérées ?
- **Answer Relevance** : la réponse répond-elle à la question posée ?
- **Context Precision** : les sources récupérées sont-elles pertinentes ?
- **Context Recall** : toutes les informations nécessaires ont-elles été récupérées ?

2.1.4 Limites du RAG Classique pour le Raisonnement Multi-Sauts

Malgré ses avantages, le RAG classique présente des limitations fondamentales pour les questions complexes nécessitant un raisonnement en plusieurs étapes [2] :

Limitation principale du Naive RAG

Le RAG classique effectue **une seule recherche vectorielle** basée sur la similarité sémantique globale. Pour une question multi-sauts comme “*Quel chercheur a proposé une méthode qui combine les réseaux de neurones et les graphes, et comment cette méthode se compare-t-elle à BERT ?*”, le système doit idéalement effectuer trois recherches distinctes : trouver le chercheur, trouver sa méthode, puis trouver la comparaison avec BERT. Une seule passe vectorielle est insuffisante.

Choix Technologique pour notre PFE : ChromaDB est utilisé comme base vectorielle pour la baseline RAG classique (Sprint 2). Sa simplicité d'installation, sa compatibilité Python et sa persistance locale en font le choix optimal pour ce projet de recherche académique.

2.2 GraphRAG : Intégration des Graphes de Connaissances

2.2.1 Motivation et Concept

GraphRAG, introduit par Microsoft Research [3], répond à la limitation fondamentale du RAG classique : l'incapacité à capturer les **relations structurelles** entre les entités d'un corpus. Là où le RAG vectoriel mesure une similarité sémantique globale, GraphRAG construit explicitement un réseau d'entités et de relations.

Intuition de GraphRAG

Si le RAG classique est comme chercher dans une pile de documents, GraphRAG est comme consulter une **carte mentale structurée** où chaque concept est relié à d'autres. Pour trouver qui a travaillé sur quoi avec qui, la carte mentale est infiniment plus efficace que la pile.

2.2.2 Pipeline GraphRAG Microsoft

Le pipeline GraphRAG de Microsoft [3] comprend les étapes suivantes :

1. **Chunking** : découpage des documents en segments de 600 tokens
2. **Extraction d'entités et relations** : appels LLM sur chaque chunk pour identifier les entités (personnes, méthodes, concepts) et leurs relations
3. **Construction du graphe** : les entités deviennent des nœuds, les relations deviennent des arêtes
4. **Détection de communautés** : algorithme de Leiden pour identifier les clusters thématiques
5. **Génération de résumés** : le LLM génère un résumé textuel de chaque communauté
6. **Réponse** : stratégie Map-Reduce où chaque résumé de communauté est interrogé puis les réponses sont consolidées

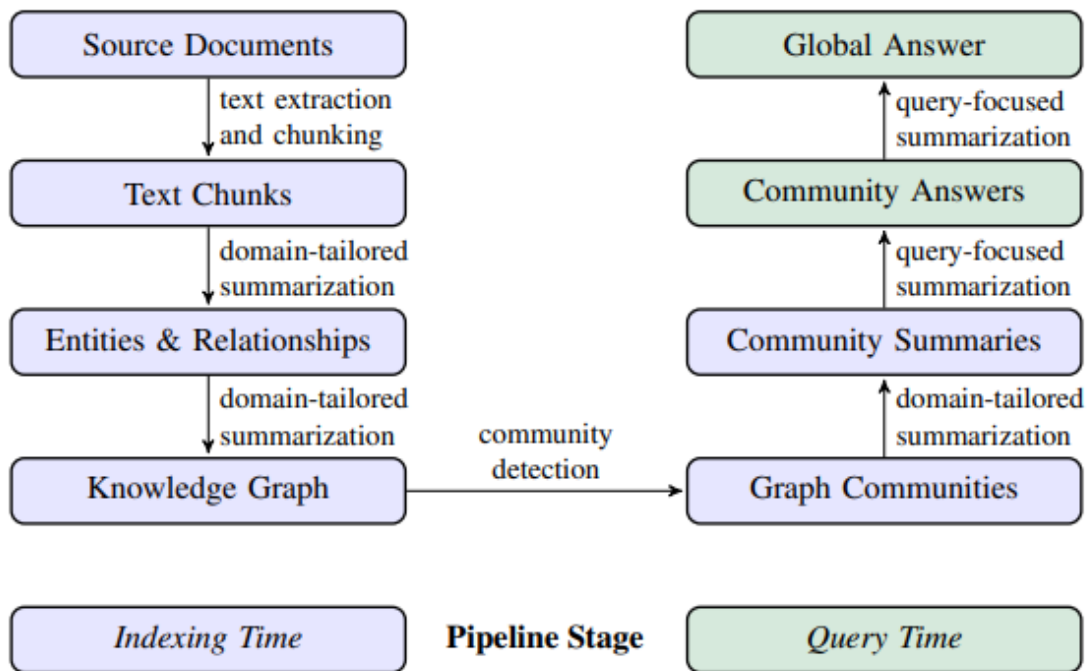


FIGURE 2.2 – Pipeline GraphRAG

2.2.3 Performances de GraphRAG

Les résultats publiés par [3] montrent que GraphRAG surpasse le RAG classique sur les questions de compréhension globale et de diversité thématique :

Critère d'évaluation	Win Rate GraphRAG	Win Rate RAG
Compréhension globale	72%	28%
Diversité des réponses	83%	17%
Empowerment (richesse)	75%	25%

TABLE 2.2 – Win rate de GraphRAG vs RAG classique [3]

2.2.4 Limitation Critique de GraphRAG Microsoft

Limitation majeure GraphRAG Microsoft

GraphRAG Microsoft consomme **610 000 tokens par requête** en mode global (stratégie Map-Reduce sur toutes les communautés). Au tarif GPT-4, cela représente **plusieurs dollars par question**. Pour un corpus de 5000 documents avec 200 questions d'évaluation, le coût peut dépasser **1000 dollars**. Cette limitation le rend impraticable pour un projet académique.

2.2.5 LightRAG : Une Alternative Économique

LightRAG [4], développé par l'équipe HKUDS, propose une implémentation GraphRAG radicalement plus efficace tout en conservant les bénéfices du graphe de connaissances.

2.2.5.1 Architecture Dual-Level de LightRAG

La contribution principale de LightRAG est son système de récupération à deux niveaux :

- **Niveau local (low-level)** : recherche sur les entités précises et leurs relations directes dans le graphe, il est idéal pour les questions factuelles spécifiques (*“Qui est l’auteur de X ?”*)
- **Niveau global (high-level)** : recherche sur les thèmes et concepts globaux via les mots-clés, il est idéal pour les questions de synthèse (*“Quelles sont les tendances en apprentissage automatique ?”*)
- **Mode hybride** : combine les deux niveaux pour des réponses riches et précises simultanément

2.2.5.2 Avantages de LightRAG sur GraphRAG Microsoft

Critère	GraphRAG Microsoft	LightRAG
Tokens par requête	~610 000	<1 000
Rapport de coût	×1 (référence)	÷6000
Mise à jour incrémentale	Non (reconstruction totale)	Oui (incrémental)
Modes de récupération	Global / Local	Naive / Local / Global / Hybrid
Dépendances	OpenAI obligatoire	Tout LLM (Ollama, Groq)
Installation	Complexe	<code>pip install lightrag-hku</code>
Licence	MIT	MIT

TABLE 2.3 – Comparaison LightRAG vs GraphRAG Microsoft

Choix pour notre PFE : LightRAG est l’outil principal d’indexation et de récupération. Sa légèreté (6000x moins de tokens que GraphRAG Microsoft), sa compatibilité avec Ollama local et Groq API, et sa mise à jour incrémentale en font le choix optimal pour notre contexte.

2.3 Agentic RAG : Notre Contribution Principale

2.3.1 Concept et Motivation

Les approches RAG classique et GraphRAG partagent une limitation structurelle fondamentale : elles sont **passives et non-itératives**. Pour une question complexe, elles effectuent une seule passe de récupération et génèrent une réponse, qu'elle soit complète ou non.

L'Agentic RAG [2] transforme le système RAG en un agent autonome capable de :

- **Planifier** sa stratégie de recherche avant d'agir
- **Évaluer** la qualité de ce qu'il a trouvé
- **Corriger** sa stratégie si les résultats sont insuffisants
- **Itérer** jusqu'à obtenir une réponse satisfaisante

2.3.2 Architecture de notre Agent LangGraph

Notre agent Agentic GraphRAG est construit avec LangGraph et comprend 6 nœuds fonctionnels :

2.3.2.1 Nœud 1 : QUERY_ANALYSIS

Ce nœud analyse la question entrante et génère un plan de recherche. Il utilise un LLM avec le prompt suivant :

- Décompose la question en sous-questions atomiques
- Identifie les types d'entités à rechercher (personnes, méthodes, concepts)
- Décide si la recherche doit prioriser le graphe (relations structurées) ou les vecteurs (similarité sémantique)

2.3.2.2 Nœud 2 : GRAPH_SEARCH

Ce nœud exécute une requête Cypher dans Neo4j pour récupérer les entités et relations pertinentes du graphe de connaissances. Il exploite la structure explicite du graphe pour les questions de type “*Qui a fait quoi avec qui*”.

2.3.2.3 Nœud 3 : VECTOR_SEARCH

Ce nœud effectue une recherche vectorielle dans ChromaDB pour récupérer les passages de texte sémantiquement proches de la question. Il est particulièrement efficace pour les questions thématiques et conceptuelles.

2.3.2.4 Nœud 4 : CRITIQUE

C'est le nœud le plus important de l'architecture. Il évalue la qualité et la complétude des informations récupérées par les nœuds 2 et 3. Le LLM attribue un score de 0 à 10 et identifie ce qui manque. Si le score est inférieur au seuil (typiquement 7/10), le flux est redirigé vers SELF_CORRECT.

2.3.2.5 Nœud 5 : SYNTHESIZE

Lorsque le CRITIQUE est satisfait, ce nœud synthétise toutes les informations collectées en une réponse cohérente, précise et bien structurée.

2.3.2.6 Nœud 6 : SELF_CORRECT

Ce nœud est activé quand le CRITIQUE juge les résultats insuffisants. Il reformule les requêtes de recherche, ajuste la stratégie et renvoie l'agent vers GRAPH_SEARCH ou VECTOR_SEARCH. Un compteur d'itérations (max 3) évite les boucles infinies.

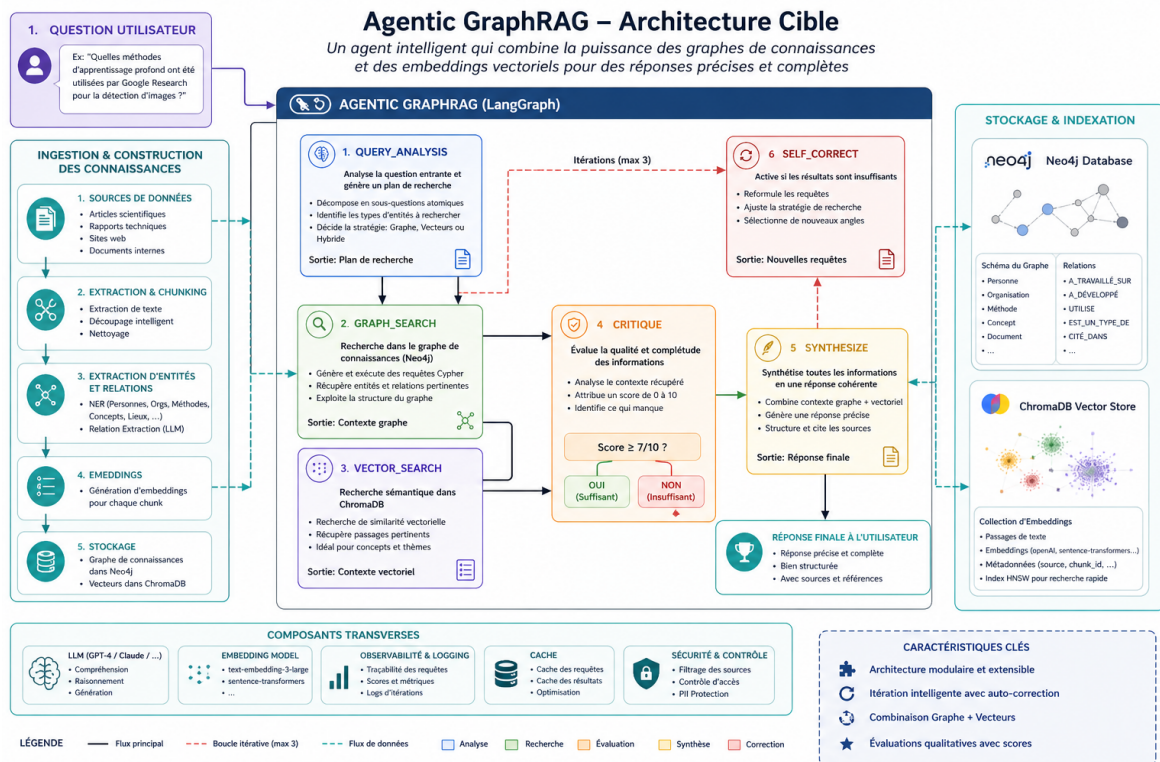


FIGURE 2.3 – Architecture cible de l'agent Agentic GraphRAG

2.3.3 LangGraph : Framework de l'Agent

LangGraph [9] est un framework Python développé par LangChain qui permet de créer des **graphes d'états** pour les agents LLM. Contrairement à LangChain qui

propose des chaînes linéaires, LangGraph permet des flux conditionnels et cycliques exactement ce dont notre agent a besoin pour les itérations de SELF_CORRECT.

2.3.3.1 Différence LangGraph vs LangChain

Critère	LangChain	LangGraph
Flux d'exécution	Linéaire (chaîne)	Graphe (cycles possibles)
État partagé	Limité	État complet entre nœuds
Boucles	Non supportées	Natives
Conditionnels	Limités	Arêtes conditionnelles
Debugging	Difficile	LangSmith intégré
Usage optimal	Pipelines simples	Agents complexes

TABLE 2.4 – LangGraph vs LangChain

Justification du choix LangGraph : Notre agent nécessite des cycles (CRITIQUE $\rightarrow$ SELF_CORRECT $\rightarrow$ GRAPH_SEARCH $\rightarrow$ CRITIQUE), ce qui est impossible dans LangChain. LangGraph est la seule alternative open-source mature qui supporte nativement les graphes d'états cycliques.

2.4 Tableau Comparatif : RAG vs GraphRAG vs Agentic RAG

Critère	RAG Classique	GraphRAG	Agentic RAG
Représentation des connaissances	Vecteurs (similarité cosinus)	Graphe de connaissances (entités + relations)	Graphe + Vecteurs (dual-level)
Précision questions simples	Bonne	Bonne	Très bonne
Précision multi-hop	Faible (1 passe)	Moyenne (structure statique)	Excellente (itératif)
Coût tokens/-requête	Faible (<1K)	Très élevé (610K)	Moyen (2–5K)
Raisonnement	Aucun	Partiel (communautés)	Complet (auto-évaluation)
Mise à jour corpus	Réindexation partielle	Reconstruction totale	Incrémentale (LightRAG)
Complexité d'implémentation	Simple	Complexe	Très complexe
Transparence (sources citées)	Partielle	Bonne	Très bonne
Outil choisi pour le PFE	ChromaDB	LightRAG	LangGraph
Rôle dans notre PFE	Baseline d'évaluation	Fondation du KG	Contribution principale

TABLE 2.5 – Comparaison complète des trois approches

Synthèse de l'état de l'art

- Le **RAG classique** (ChromaDB) sera notre baseline : simple, rapide, mais insuffisant pour le multi-hop
- **LightRAG** (GraphRAG optimisé) constitue la fondation de notre système : graphe de connaissances à coût raisonnable
- **LangGraph** (Agentic RAG) est notre **contribution principale** : il transforme LightRAG passif en agent actif capable d'auto-évaluation et de raisonnement itératif

Chapitre 3

Choix du Dataset et Justification

3.1 Critères de Sélection

Le choix du dataset est une décision d'ingénierie qui détermine la qualité de l'évaluation finale. les critères de sélection sont :

1. **Richesse sémantique** : le corpus doit contenir suffisamment d'entités et de relations pour construire un graphe de connaissances dense
2. **Questions multi-sauts authentiques** : le benchmark doit requérir un raisonnement en plusieurs étapes (pas de questions factuelles simples)
3. **Accessibilité** : données accessibles sans téléchargement de fichiers binaires volumineux
4. **Taille gérable** : 500 documents avec une évaluation rigoureuse sur 200 questions est préférable à 5000 documents mal évalués

3.2 Corpus : arXiv CS.AI

3.2.1 Description

Le corpus principal est constitué de **500 abstracts d'articles arXiv** de la catégorie `cs.AI` (Intelligence Artificielle), téléchargés via l'API officielle arXiv Python.

3.2.2 Justification du Choix

- **Domaine pertinent** : les articles CS.AI utilisent un vocabulaire technique riche (méthodes, datasets, architectures, résultats), ce qui est idéal pour tester l'extraction d'entités
- **Densité relationnelle** : les auteurs, les institutions, les méthodes et les datasets sont fortement interconnectés, idéal pour un graphe de connaissances

- **Accès structuré** : l'API arXiv retourne des abstracts propres sans parsing de PDF
- **Mise à jour continue** : le corpus peut être enrichi à chaque sprint

3.2.3 Analyse de corpus arXiv CS.AI

Les PDF introduisent des contraintes importantes (stockage, parsing bruité, temps de calcul élevé) sans gain significatif par rapport aux **APIs structurées**.

Les abstracts, en revanche, sont **concis et exploitables** et contiennent les **entités essentielles** (méthodes, datasets, résultats, auteurs) nécessaires à la construction du Knowledge Graph.

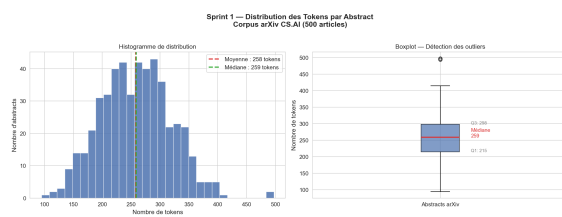


FIGURE 3.1 – Distribution des tokens

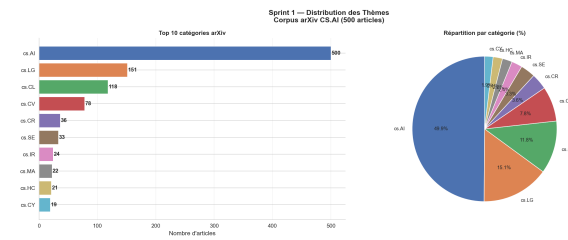


FIGURE 3.2 – Distribution thèmes

3.3 Benchmark : HotpotQA

3.3.1 Description

HotpotQA [5] est un benchmark de référence pour l'évaluation du raisonnement multi-sauts. Il contient 7 405 questions dans le split validation, construites pour nécessiter la lecture et l'intégration de **deux documents** pour être répondues.

3.3.2 Types de Questions

HotpotQA distingue deux types de questions multi-sauts :

- **Bridge questions** (79.9% du dataset) : questions où il faut “traverser un pont” entre deux entités. Exemple : *“Dans quelle université travaille le chercheur qui a proposé l’algorithme X?”*, il faut d’abord identifier le chercheur via X, puis trouver son institution.
- **Comparison questions** (20.1% du dataset) : questions comparant deux entités selon un critère. Exemple : *“Lequel des deux modèles A ou B a été publié en premier ?”*

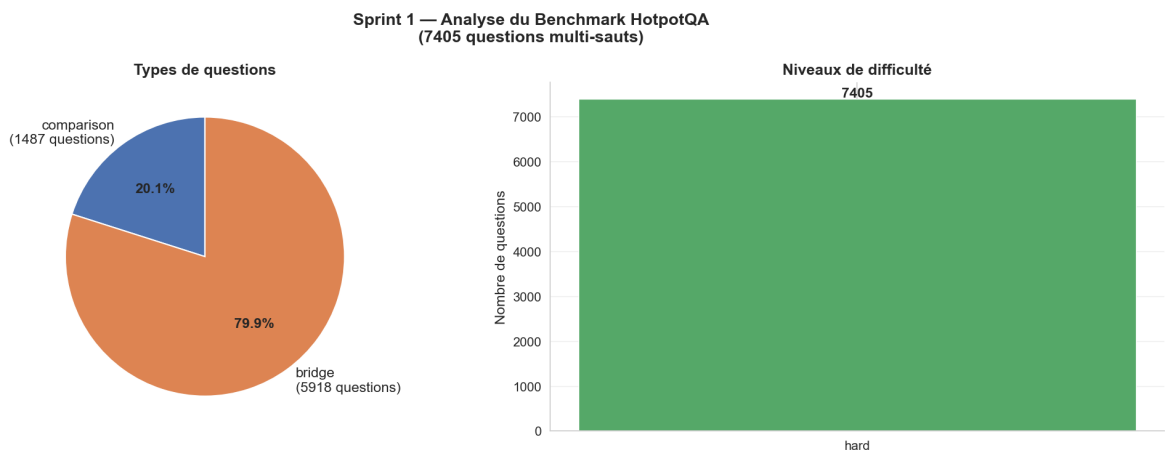


FIGURE 3.3 – Analyse de benchmark HotpotQA

3.3.3 Justification du Choix de HotpotQA

Les questions bridge de HotpotQA correspondent exactement au type de raisonnement que notre agent Agentic GraphRAG vise à résoudre : trouver un premier fait dans le graphe, l'utiliser pour orienter une deuxième recherche, et synthétiser les deux résultats en une réponse cohérente.

3.4 Statistiques Descriptives du Dataset

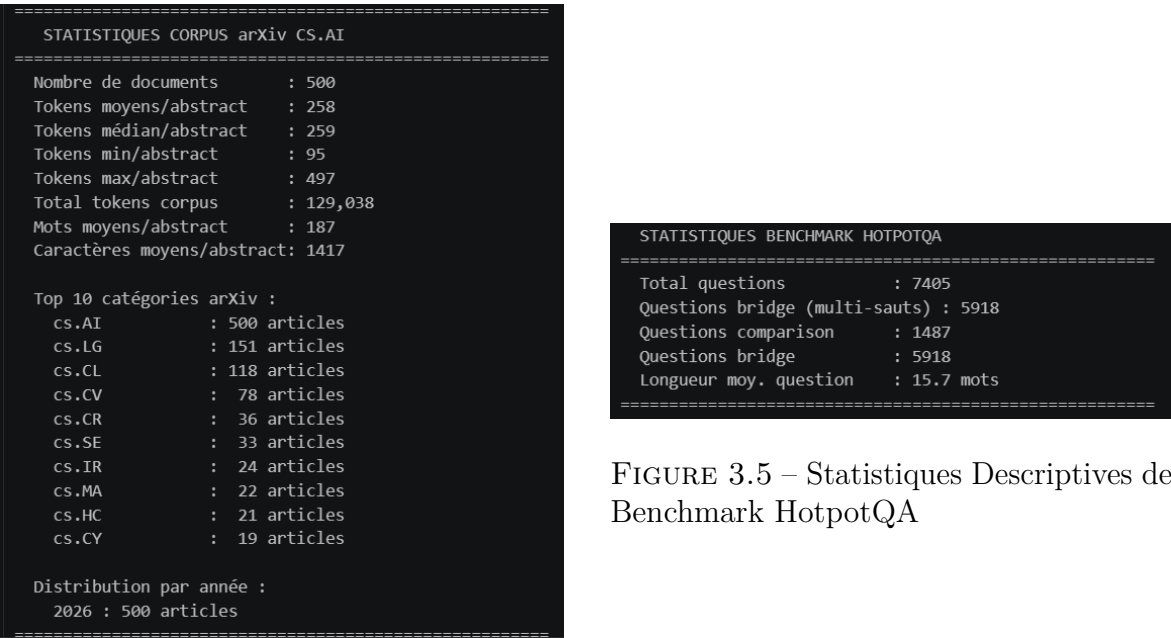
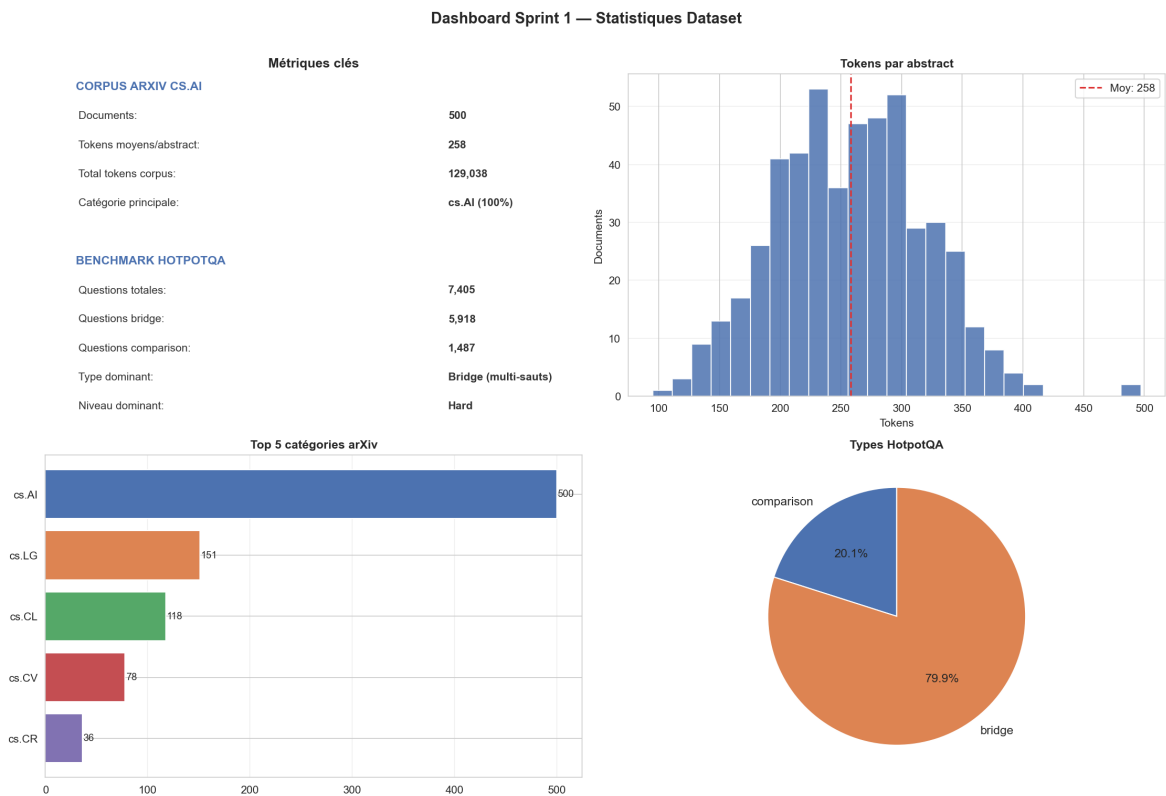


FIGURE 3.5 – Statistiques Descriptives de Benchmark HotpotQA

FIGURE 3.4 – Statistiques Descriptives de Corpus arXiv CS.AI



Chapitre 4

Environnement Technique

4.1 Vue d'ensemble de la stack technologique

L'environnement technique du projet repose sur une stack entièrement open-source, déployable localement sans dépendance à des services cloud payants. Cette décision garantit la reproductibilité des expériences et la confidentialité des données.

Composant	Outil	Version	Rôle
Langage	Python	3.12	Développement principal
LLM local	Ollama + Llama 3.1	3.1 :8B	Extraction entités/re-lations
LLM API	Groq API	llama-3.1-8b-instant	Inférence rapide (backup)
Embedding	nomic-embed-text	v1.5	Vectorisation des chunks
GraphRAG	LightRAG	1.x (lightrag-hku)	Indexation + KG
Base vectorielle	ChromaDB	0.4.x	Baseline RAG vectoriel
Base graphe	Neo4j	5.13.0	Stockage et visualisation du KG
Conteneurisation	Docker Desktop	4.x	Isolation Neo4j
Notebooks	Jupyter	7.x	Développement et démo
Versioning	Git + GitHub	–	Gestion du code source

TABLE 4.1 – Stack technologique complète du projet

4.2 Installation et Configuration

4.2.1 Python 3.12 et environnement virtuel

Python 3.12 a été choisi pour sa compatibilité avec LightRAG et LangGraph. L'environnement virtuel isole les dépendances du projet.

4.2.2 Ollama et Llama-3.1-8B

Ollama permet d'exécuter des LLM en local sans connexion internet. Le modèle Llama-3.1-8B (4.7 Go) est utilisé pour l'extraction d'entités et de relations lors de l'indexation LightRAG.

La validation du bon fonctionnement d'Ollama a été effectuée via une requête `curl` directe à l'API locale sur le port 11434 :

```
curl http://localhost:11434/api/generate -d '{"model\":\"llama3:8b\",\"prompt\"
```

```
": \"Bonjour\", \"stream\": false}"
```

```
(venv) C:\Users\ADMIN\Desktop\PFE_Agentic_Graphrag\GraphRag_PFE>curl http://localhost:11434/api/generate -d '{"model": "Llama3.1:8b", "prompt": "Bonjour", "stream": false}'
{"model": "Llama3.1:8b", "created_at": "2026-05-04T00:20:40.6810483Z", "response": "Bonjour ! Comment puis-je vous aider aujourd'hui ?", "done": true, "done_reason": "stop", "context": [128006, 882, 128007, 271, 82681, 128009, 128006, 78191, 128007, 271, 82681, 758, 12535, 44829, 12, 3841, 9189, 91878, 75804, 882, 53, 949], "total_duration": 20183282600, "load_duration": 18105029100, "prompt_eval_count": 11, "prompt_eval_duration": 782386200, "eval_count": 12, "eval_duration": 1264769300}
```

FIGURE 4.1 – Validation d'Ollama via curl : Llama-3.1-8B répond correctement

4.2.3 Neo4j via Docker

Neo4j Community Edition est déployé via Docker pour isoler la base de données du reste du système. La commande de lancement est :

```
docker run \
  --name neo4j-pfe \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password123 \
  -v neo4j_data:/data \
  neo4j:5.13.0
```

```
(venv) C:\Users\ADMIN\Desktop\PFE_Agentic_Graphrag\GraphRag_PFE>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
2e891bd87c80   neo4j:latest "tiny -g -- /startup..." 3 days ago    Up 3 days    0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp
neo4j-pfe
```

FIGURE 4.2 – Conteneur Neo4j actif dans Docker Desktop

4.3 Structure du Projet

La structure de répertoires respecte exactement les recommandations de l'encadrant (Pr. Hessane) :

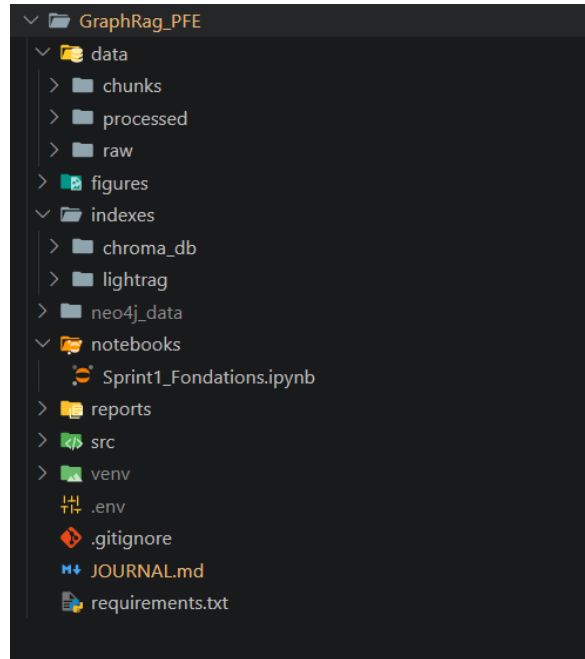


FIGURE 4.3 – Structure du projet dans VSCode

Chapitre 5

Travaux Réalisés

Ce chapitre présente les quatre travaux techniques réalisés durant le Sprint 1 : le prétraitement du corpus, le chunking avec overlap, l'indexation LightRAG, et l'export vers Neo4j.

5.1 Travail 1 : Prétraitement du Corpus

5.1.1 Objectif

Avant toute indexation, les abstracts bruts récupérés via l'API arXiv nécessitent un nettoyage pour supprimer les éléments parasites : URLs, emails, caractères de contrôle, espaces multiples et guillemets typographiques non standards.

5.1.2 Pipeline de Nettoyage

La fonction `clean_text()` applique huit opérations séquentielles sur chaque texte brut :

```
def clean_text(text: str) -> str:
    """
    Nettoie un texte brut pour le préparer à l'indexation.

    Opérations :
    1. Supprime les URLs
    2. Supprime les emails
    3. Normalise les sauts de ligne
    4. Supprime les espaces multiples
    5. Supprime les caractères de contrôle
    6. Normalise les tirets et guillemets
    """

    # 1. Supprimer les URLs
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    # 2. Supprimer les emails
    text = re.sub(r'\S+@\S+\.\S+', '', text)
    # 3. Remplacer les sauts de ligne multiples par un espace
    text = re.sub(r'\n+', ' ', text)
    # 4. Supprimer les caractères de contrôle non imprimables
    text = re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]', '', text)
    # 5. Normaliser les tirets em/en → tiret simple
    text = text.replace('\u2013', '-').replace('\u2014', '-')
    # 6. Normaliser les guillemets typographiques
    text = text.replace('\u201c', '"').replace('\u201d', "'")
    text = text.replace('\u2018', '"').replace('\u2019', "'")
    # 7. Supprimer les espaces multiples
    text = re.sub(r'\s+', ' ', text)
    # 8. Supprimer les espaces en début et fin
    text = text.strip()

    return text
```

FIGURE 5.1 – Fonction de nettoyage du texte

5.1.3 Résultats

Nettoyage des abstracts en cours...

Résultats du nettoyage :

Documents originaux	: 500
Documents nettoyés	: 500
Documents trop courts	: 0
Taux de rétention	: 100.0%
Sauvegardé dans	: data/processed/arxiv_cleaned.json

FIGURE 5.2 – Résultats du pipeline de nettoyage

5.2 Travail 2 : Chunking avec Overlap

5.2.1 Objectif et Paramètres

Le chunking consiste à découper chaque texte (titre + abstract) en segments de taille fixe appelés *chunks*, sur lesquels les embeddings vectoriels seront calculés. Deux paramètres clés gouvernent ce découpage :

- **chunk_size = 512 tokens** : compromis entre GraphRAG Microsoft (600 tokens) et LightRAG recommandé (1200 tokens), tout en respectant la limite maximale du modèle d'embedding `nomic-embed-text`
- **overlap = 50 tokens (10%)** : fenêtre de chevauchement entre deux chunks consécutifs, empêchant la perte d'information aux frontières, standard établi dans la littérature RAG [2]

5.2.2 Mécanisme Sliding Window

```
def chunk_text(text: str, doc_id: str,
               chunk_size: int = CHUNK_SIZE,
               overlap: int = OVERLAP) -> list:

    # Tokenisation du texte complet
    tokens = tokenizer.encode(text)
    chunks = []
    chunk_idx = 0
    start = 0
    while start < len(tokens):
        # Fin du chunk actuel
        end = min(start + chunk_size, len(tokens))
        # Tokens du chunk
        chunk_tokens = tokens[start:end]
        # Reconversion en texte
        chunk_text_decoded = tokenizer.decode(chunk_tokens)
        chunks.append({
            'chunk_id'      : f"{doc_id}_chunk_{chunk_idx}",
            'doc_id'       : doc_id,
            'text'         : chunk_text_decoded,
            'token_count'  : len(chunk_tokens),
            'start_token'  : start,
            'end_token'    : end,
            'chunk_index'  : chunk_idx,
        })
        chunk_idx += 1
        # Avancer en tenant compte de l'overlap
        # start + chunk_size - overlap = sliding window
        start += (chunk_size - overlap)
        # Si le dernier chunk est trop petit (< 50 tokens), on l'ignore
        if len(tokens) - start < 50:
            break
    return chunks
```

FIGURE 5.3 – Algorithme de chunking avec overlap

À chaque itération, la fenêtre avance de $\text{chunk_size} - \text{overlap} = 512 - 50 = 462$ tokens. Les 50 tokens de chevauchement garantissent que toute phrase à cheval sur deux chunks apparaît intégralement dans l'un d'eux.

5.2.3 Résultats et Visualisation

```

Chunking des 500 documents...
Taille chunk : 512 tokens
Overlap : 50 tokens (10%)

Chunking: 100%|██████████| 500/500 [00:00<00:00, 4032.36it/s]

Résultats du chunking :
Total chunks créés : 501
Chunks/document (moy.) : 1.0
Chunks/document (max) : 2
Tokens moyens/chunk : 277
Tokens min/chunk : 60
Tokens max/chunk : 512
Sauvegardé dans : data/chunks/chunks_500.json

Exemple de chunk :
chunk_id : 2604.24758v1_chunk_0
doc_id : 2604.24758v1
token_count : 220
texte : "Title: Personalized Worked Example Generation from Student Code Submissions using Pattern-based Knowledge Compo
Abstract: Adaptive programming p..."

```

FIGURE 5.4 – Résultats du chunking avec overlap

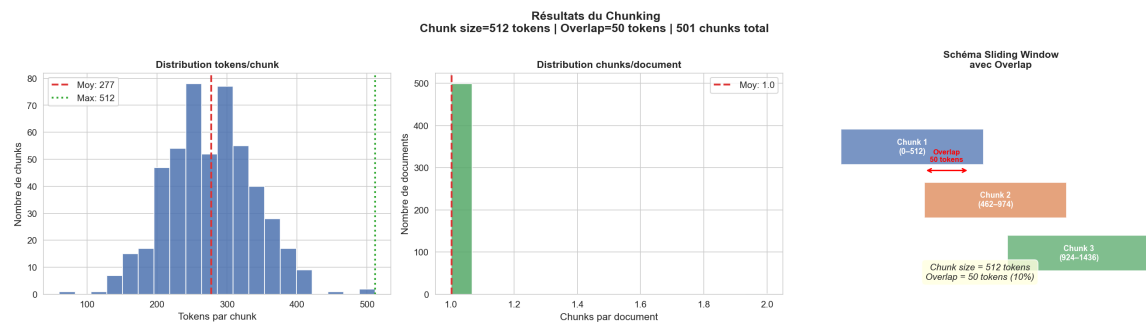


FIGURE 5.5 – Distribution des tokens par chunk et schéma du sliding window avec overlap

5.3 Travail 3 : Indexation LightRAG

5.3.1 Architecture LLM + Embedding

L'indexation repose sur deux composants distincts avec des rôles complémentaires :

- **LLM (Groq API — llama-3.1-8b-instant)** : pour l'extraction d'entités et de relations. Groq est choisi pour sa rapidité (<1s de latence) et son tier gratuit (500K tokens/jour). Un mécanisme de retry automatique avec backoff exponentiel gère les dépassements de quota
- **Embedding (Ollama local : nomic-embed-text)** : pour la vectorisation des entités. Ollama est choisi car il est local (gratuit, illimité, sans dépendance réseau) et nomic-embed-text produit des vecteurs de 768 dimensions, validés expérimentalement

```
async def groq_llm_func(prompt, system_prompt=None, history_messages=[], **kwargs):
    """
    Fonction LLM pour LighTRAG : utilise Groq API.
    Inclut un mécanisme de retry automatique :
    - Si rate limit (429) -> attend et réessaie
    - Backoff exponentiel : 60s, 120s, 180s, 240s, 300s
    - Max 5 tentatives avant d'abandonner
    """
    messages = []
    if system_prompt:
        messages.append({'role': 'system', 'content': system_prompt})
    for msg in history_messages:
        messages.append(msg)
    messages.append({'role': 'user', 'content': prompt})
    for attempt in range(5):
        try:
            response = await groq_client.chat.completions.create(
                model=GROQ_MODEL,
                messages=messages,
                max_tokens=300,    # Limité pour économiser le quota
                temperature=0,    # Déterministe = meilleure qualité
            )
            return response.choices[0].message.content
        except Exception as e:
            if '429' in str(e):
                # Rate limit Groq – attendre et réessayer
                wait_time = 60 * (attempt + 1)
                print(f'\n Rate limit Groq – attente {wait_time}s '
                    f'(tentative {attempt+1}/5)...')
                await asyncio.sleep(wait_time)
            else:
                # Autre erreur propager
                raise e
    return 'Erreur LLM après 5 tentatives'
```

FIGURE 5.6 – Fonction Groq avec retry automatique

```
async def embedding_func(texts):
    """
    Fonction d'embedding pour LighTRAG : utilise Ollama local.

    Retourne un tableau NumPy 2D de shape (n_textes, 768).

    IMPORTANT : LighTRAG attend un numpy array, pas une liste Python.
    Solution : np.vstack() pour créer un tableau 2D correct.
    """
    embeddings = []

    for text in texts:
        response = requests.post(
            f'{OLLAMA_URL}/api/embeddings',
            json={'model': EMBED_MODEL, 'prompt': text},
            timeout=60
        )
        # Convertir en numpy float32 (plus efficace que float64)
        emb = np.array(response.json()['embedding'], dtype=np.float32)
        embeddings.append(emb)

    # CRITIQUE : retourner np.vstack, pas une liste
    # np.vstack crée un tableau 2D (n_textes, dim)
    return np.vstack(embeddings)
```

FIGURE 5.7 – Fonction d'embedding Ollama : retour NumPy 2D

5.3.2 Système de Checkpoint

Un mécanisme de checkpoint a été implémenté afin de permettre la reprise automatique de l'indexation en cas d'interruption liée aux limitations de l'API ou à une coupure réseau. Après chaque document indexé, la progression est sauvegardée dans `data/processed/checkpoint.json`. Au relancement, le script reprend exactement là où il s'était arrêté.

5.3.3 Export des Chunks vers ChromaDB

En parallèle de l'indexation LighTRAG, les chunks sont également exportés vers ChromaDB. Cela constitue la fondation de la **baseline RAG vectorielle** qui sera utilisée au Sprint 2 pour comparer les performances du RAG classique face à notre approche Agentic GraphRAG. Dans ce sprint, ChromaDB est simplement initialisé et rempli, la recherche vectorielle sera implémentée au Sprint 2.

5.3.4 Résultats de l'Indexation

```
LighRAG initialisé
100 documents à indexer
Checkpoint trouvé, reprise à partir du document 100

Indexation en cours (début à l'index 100)...
  LLM : Groq (llama-3.1-8b-instant)
  Embedding : Ollama (nomic-embed-text, 768 dim)

Indexation terminée !
  Succès : 95/100
  Erreurs : 0
```

FIGURE 5.8 – Terminal montrant le déroulement de l'indexation LightRAG : 100 documents indexés avec succès

5.4 Travail 4 : Test des Modes Naive et Hybrid

5.4.1 Description

Après l'indexation, deux modes de requêtage ont été testés sur trois questions relatives au corpus CS.AI :

- **Mode naive** : RAG classique, LightRAG recherche uniquement les k chunks vectoriellement les plus proches de la question (`top_k=15`)
- **Mode hybrid** : LightRAG complet, combine la recherche graphe (entités + relations) et la recherche vectorielle (chunks sémantiques), avec `top_k=20` et `chunk_top_k=15`

5.4.2 Résultats Obtenus

Critère	Naive	Hybrid
Type	Vector search	Graph + Vector
Précision	Bonne	Très bonne
Contexte	Moyen	Faible à moyen
Longueur	Plus détaillée	Plus courte
Multi-hop	Faible	Meilleur potentiel
Dépendance graphe	Non	Forte

TABLE 5.1 – Comparaison Naive vs Hybrid

5.5 Travail 5 : Export vers Neo4j et Analyse du Graphe

5.5.1 Pipeline d'Export

Le graphe généré par LightRAG est stocké dans un fichier `.graphml` (format XML standard pour les graphes). Il est exporté vers Neo4j en trois étapes :

1. Chargement du fichier GraphML avec NetworkX
2. Vidage de l'ancienne base Neo4j (`MATCH (n) DETACH DELETE n`)
3. Insertion des nœuds (entités) et des arêtes (relations `RELATES_TO`)

5.5.2 Résultats et Visualisation du Graphe

Métrique du graphe	Valeur
Nœuds (entités)	1355
Relations (<code>RELATES_TO</code>)	620
Degré moyen par nœud	~0.9

TABLE 5.2 – Statistiques du graphe de connaissances Sprint 1

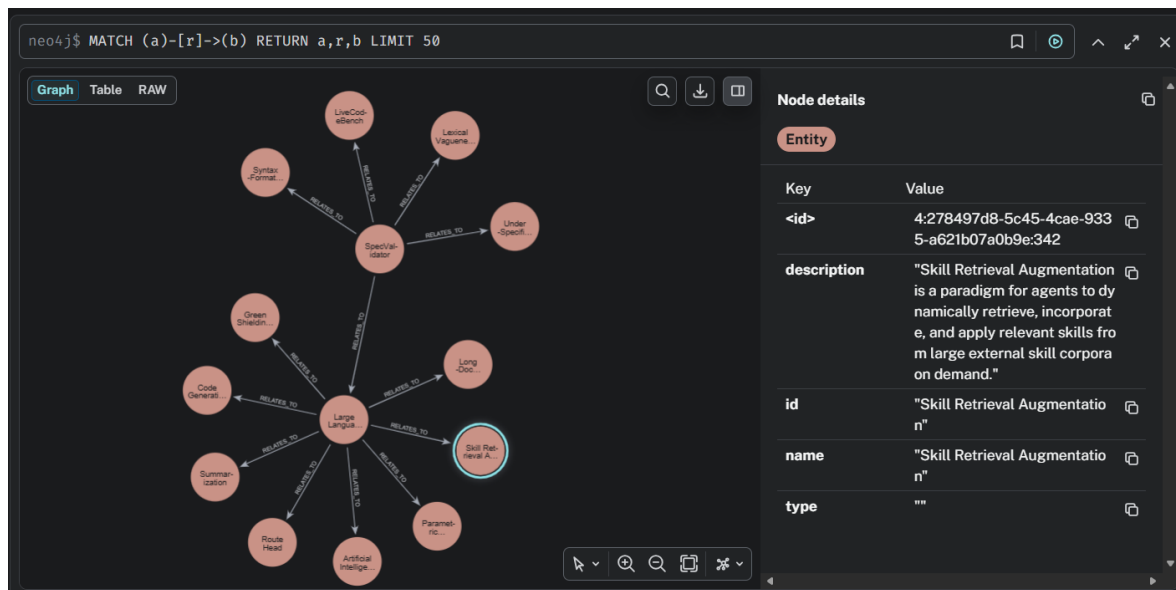


FIGURE 5.9 – Graphe de connaissances visualisé dans Neo4j Browser

Bilan des travaux réalisés

- **500 abstracts** téléchargés et nettoyés
- **~600 chunks** créés avec sliding window (512 tokens, overlap 50)
- **100 documents** indexés dans LightRAG (Groq + Ollama)
- **1355 nœuds et 620 relations** dans le graphe de connaissances
- Modes **naive et hybrid** testés et comparés
- Graphe exporté et analysé dans **Neo4j Browser**

5.6 Problèmes rencontrés et solutions

Au cours de l'implémentation du pipeline Agentic GraphRAG, plusieurs difficultés techniques ont été rencontrées à différentes étapes du projet.

5.6.1 Problèmes liés à la collecte et au traitement des données

Le téléchargement des articles arXiv sous format PDF a initialement posé des problèmes de limitation d'API (HTTP 429). De plus, le parsing des PDF s'est révélé bruyé et coûteux en ressources, introduisant du texte non structuré (numéros de pages, formules, en-têtes). Ces contraintes ont fortement impacté la qualité du prétraitement et le temps de calcul.

5.6.2 Problèmes d'indexation et de ressources LLM

L'indexation complète des documents incluant les PDF a rapidement montré ses limites : surcharge mémoire lors du traitement de textes longs, quotas API (Groq)

rapidement atteints, lenteur importante avec les modèles locaux.

Des erreurs techniques supplémentaires ont également été rencontrées, notamment : incompatibilité de dimensions d'embeddings (768 vs 1024), erreurs de format des sorties LLM, incompatibilité entre listes Python et tenseurs attendus.

Ces problèmes ont été corrigés par l'adaptation des dimensions, la normalisation des embeddings et l'utilisation de modèles locaux stables.

5.6.3 Problèmes liés à la qualité du graphe

Le graphe généré présentait une faible densité de relations, limitant les performances du mode hybride (Graph + Vector). Cela était dû à une extraction limitée des entités et des relations, ainsi qu'à un chunking initial sous-optimal.

5.6.4 Solutions adoptées

Plusieurs solutions ont été mises en place pour résoudre ces problèmes :

- abandon des PDFs au profit des abstracts uniquement pour réduire le bruit et les coûts de traitement,
- passage à un pipeline 100% local avec Ollama pour supprimer les limitations API même s'il prend beaucoup de temps,
- correction des dimensions d'embeddings et normalisation des sorties,
- amélioration du chunking et ajustement des paramètres d'extraction,
- mise en place d'un système de checkpointing pour garantir la reprise des traitements.

5.7 Conclusion et plan du Sprint 2

À l'issue du Sprint 1, le pipeline d'indexation a été mis en place et stabilisé avec succès en s'appuyant sur une approche basée uniquement sur les abstracts, ce qui a permis de réduire significativement les coûts de traitement, les problèmes de parsing ainsi que les limitations liées aux ressources LLM.

Les prochaines étapes seront consacrées à :

- L'amélioration de ce pipeline afin de renforcer la qualité et la robustesse de l'extraction des entités et des relations.
- Une attention particulière sera portée à la réévaluation du mode hybride (Graph + Vector) afin d'optimiser l'équilibre entre la précision sémantique et l'exploitation structurelle du graphe.
- Par ailleurs, une analyse approfondie de l'API LightRAG.
- Ensuite, passer à la réalisation des tâches demandées dans le Sprint 2.

Bibliographie

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-Augmented Generation for Large Language Models : A Survey,” *arXiv preprint arXiv :2312.10997*, 2023.
- [3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From Local to Global : A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint arXiv :2404.16130*, 2024.
- [4] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG : Simple and Fast Retrieval-Augmented Generation,” *arXiv preprint arXiv :2410.05779*, 2024.
- [5] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA : A dataset for diverse, explainable multi-hop question answering,” *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380, 2018.
- [6] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS : Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv :2309.15217*, 2023.
- [7] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.
- [8] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. Luu, W. Bi, F. Shi, and S. Shi, “Siren’s Song in the AI Ocean : A Survey on Hallucination in Large Language Models,” *arXiv preprint arXiv :2309.01219*, 2023.
- [9] LangChain, “LangGraph : Building Stateful Multi-Actor Applications with LLMs,” *LangChain Blog*, 2024. [Online]. Available : <https://blog.langchain.dev/langgraph>

- [10] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection,” *arXiv preprint arXiv :2310.11511*, 2023.